



WEB - Hipermédia e Acessibilidades

SESSÃO 4

Pedro Mendonça

CONTEÚDOS

- Introdução ao Javascript
- **Princípios básicos de JS**
- Funções Avançadas
- Javascript assíncrono
- Fetch API

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- Eventos
 - Um evento HTML pode ser algo que o browser ou utilizador faz;
 - Exemplos de eventos:
 - Uma página da web HTML terminou de carregar
 - Introdução de valores num input
 - Click de um botão
 - Se executar window no modo consola da browser poderá aceder à listagem de eventos disponíveis no browser. Eventos mais comuns: onchange; onclick; onload; onsubmit;
 - Podemos adicionar comportamentos aos elementos HTML utilizando o método `addEventListener`.

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- addEventListener

- Sintaxe:

- addEventListener(type, listener)
 - addEventListener(type, listener, options)
 - addEventListener(type, listener, useCapture)

- Exemplos:

- Neste caso comentámos o form uma vez que o comportamento por defeito deste é enviar os dados para uma página (target)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Eventos</title>
</head>
<body>
  <h1>Inserir Pessoa</h1>
  <!-- <form class="form"> -->
    <p>
      <label for="nome">Nome</label>
      <input type="text" id="nome">
    </p>
    <p>
      <button id="registarBTN">Registar</button>
    </p>
  <!-- </form> -->
  <script src="eventos.js"></script>
</body>
</html>
```

Princípios básicos da linguagem

- **addEventListener**

- Podemos definir um evento quando clicamos no botão:
- Podemos definir um evento quando escrevemos no input e imprime na consulta o texto digitado:
- Podemos definir um evento quando o cursor do rato passa por cima ou sai da área do botão:

```
const registrarBTN = document.getElementById('registrarBTN');  
//const registrarBTN = document.querySelector('#registrarBTN');  
  
registrarBTN.addEventListener('click', function (e) {  
    console.log("Click");  
});
```

```
const textInput = document.getElementById('nome');  
  
textInput.addEventListener('keyup', function (e) {  
    console.log(e.target.value);  
});  
  
nomeTXT.addEventListener('input', function() {  
    console.log(nomeTXT.value)  
});
```

```
registrarBTN.addEventListener('mouseover', function (e) {  
    console.log("Mouse Over");  
});  
  
registrarBTN.addEventListener('mouseout', function (e) {  
    console.log("Mouse Out");  
});
```

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- `addEventListener`
 - Obviamente quando temos um formulário necessitamos de adicionar um form;
 - Para evitar que este tenha o comportamento por defeito devemos utilizar o método `preventDefault()`.
 - Vamos criar um objeto para guardar o nosso form.

```
const form = document.querySelector('.form');  
  
form.addEventListener('submit', function (e) { // e = event  
  e.preventDefault();  
  const nome = form.querySelector('#nome').value;  
  console.log(nome);  
});
```

Princípios básicos da linguagem

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Eventos</title>
</head>
<body>
  <h1>Inserir Pessoa</h1>
  <form action="umapaginaq.html" class="form">
    <p>
      <label for="nome">Nome</label>
      <input type="text" id="nome" required>
    </p>
    <p>
      <button type="submit"
id="registar">Registar</button>
    </p>
  </form>
  <script src="ex1.js"></script>
</body>
</html>
```

```
const registarBTN = document.getElementById('registar');
const nomeTXT = document.getElementById('nome');
const form = document.querySelector('.form');
registarBTN.disabled = true;

form.addEventListener('submit', function(e) {
  e.preventDefault();
  const nome = nomeTXT.value;
  if(nome === '') {
    alert('Nome não pode ser vazio');
    return;
  }
  else {
    alert('Nome: ' + nome);
  }
});

nomeTXT.addEventListener('input', function() {
  registarBTN.disabled = nomeTXT.value === '';
});
```

Princípios básicos da linguagem

- Crie um programa de acordo com as seguintes instruções: (ex_9)
 - Crie uma página HTML com a estrutura normal. No título digite “Exercício 9”;
 - Adiciona um h1 com o texto Inserir Pessoa
 - No final da tag body adiciona um link para o ficheiro ex_9.js;
 - A página deverá ter o seguinte aspeto visual:

Inserir Pessoa

Nome:

Apelido:

Peso (kg):

Altura (cm):

Princípios básicos da linguagem

- Crie um programa de acordo com as seguintes instruções: (ex_9)
 - Quando clicar em Registrar deve surgir numa nova área os dados inseridos;
 - Crie um array de pessoas e adicione novas pessoas;
 - Após clicar no registrar os inputs devem ser limpos;
 - À medida que as pessoas forem adicionadas estas devem surgir na listagem.

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- Object date

- Documentação:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

```
const data = new Date(); // data atual
console.log(data.toString())
// ordem argumentos: a m d h M s ms
const dataPersonal = new Date(2024, 1, 1, 12, 13, 56);
const dataPersonalV2 = new Date("2024-01-01 12:13:56");
console.log(dataPersonalV2.toString())
```

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- Object date

```
// buscar elementos da data
console.log(`Dia: ${dataPersonalV2.getDate()}`)
console.log(`Mês: ${dataPersonalV2.getMonth()}`)
console.log(`Ano: ${dataPersonalV2.getFullYear()}`)
console.log(`Hora: ${dataPersonalV2.getHours()}`)
console.log(`Minuto: ${dataPersonalV2.getMinutes()}`)
console.log(`Segundo: ${dataPersonalV2.getSeconds()}`)
console.log(`msegundos:
${dataPersonalV2.getMilliseconds()}`)
// 0 Domingo ... 6 Sábado
console.log(`Dia da semana: ${dataPersonalV2.getDay()}`)
```

Princípios básicos da linguagem

- Crie um programa de acordo com as seguintes instruções: (ex_10)
 - Inicialize uma data com um valor a seu gosto;
 - Crie uma função que devolva o valor da data no formato:
Segunda feira, 12 de Fevereiro de 2024

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- Desestruturação Arrays
- Desestruturação Objetos

```
const arr = [1,2,3,4,5]
const [n1, n2, ...resto] = arr;
// n1=1 n2=2 resto = [3,4,5]
```

```
const pessoa = {
  nome : 'Carla',
  idade : 23,
  morada : {
    rua : "Rua da esquerda",
    numero: 2,
    cod_postal: "2000-005",
    localidade: "Santarém"
  }
}
const {nome, morada: {localidade}} = pessoa
console.log(nome, localidade) //Carla Santarém
```

Princípios básicos da linguagem

- Tratamento de erros

```
try {  
    nonexistentFunction();  
} catch (error) {  
    console.log("Função não existe");  
    //console.error(error);  
}
```

```
try {  
    console.log(a);  
} catch (err) {  
    console.log(`Erro: ${err}`)  
}
```

```
const a = "texto";  
if (!a) throw new Error("a variável a não está definida");  
else if(typeof a !== 'number') throw new TypeError("a variável a tem que ser number");  
else console.log(a);
```

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- setInterval

```
setInterval(() => {  
    console.log(new Date().toLocaleTimeString('pt-PT',  
    { hour12: false}));  
}, 1000)
```

```
// Criar um timer que imprime um contador de 1 em 1  
segundo  
let count = 0  
setInterval(() => {  
    count++;  
    console.log(count);  
}, 1000);
```

Princípios básicos da linguagem



Centro para o Desenvolvimento
de Competências Digitais

- setTimeout

```
// Criar um timer que imprime um contador de 1  
em 1 segundo  
let count = 0  
const timer = setInterval(() => {  
    count++;  
    console.log(count);  
}, 1000);  
  
// Parar o timer após 10 segundos  
setTimeout(() => {  
    clearInterval(timer);  
    console.log('Timer stopped');  
}, 5000);
```


CONTEÚDOS

- Introdução ao Javascript
- Princípios básicos de JS
- **Funções Avançadas**
- Javascript assíncrono
- Fetch API

Funções avançadas

- Podemos passar N parâmetros e não ter a definição desses argumentos na função:
 - Temos acesso a uma variável **arguments** que guarda um objeto de “argumentos”;
 - Funciona apenas quando usamos a definição function;

```
function multiple() {  
    console.log(arguments)  
    console.log(arguments[1])  
}  
  
multiple(2,3,4,5,6)  
multiple("teste", true, 1, 2.3)
```

```
[Arguments] { '0': 2, '1': 3, '2': 4, '3': 5, '4': 6 }  
[Arguments] { '0': 'teste', '1': true, '2': 1, '3': 2.3 }
```

Funções avançadas

- Podemos passar N parâmetros e não ter a definição desses argumentos na função:
 - Para adaptar a uma arrow function devemos usar o **operador rest (...)**

```
const multiplos = (...args) => {  
  for(item of args) {  
    console.log(item);  
  }  
}  
  
multiplos(1,2,3,4,5,6,7,8);
```

Funções avançadas

- Podemos adicionar valores por defeito

```
function soma(n1, n2, n3=23) {  
    return n1+n2+n3;  
}  
console.log(soma(2,3));  
console.log(soma(2,3,4));
```

- Destruturação

```
const val = [1,2,3,4]  
  
const [n1, n2, n3, n4] = val;  
console.log(n2)
```

```
function printNotas([nota1, nota2, nota3]) {  
    console.log(`Nota1: ${nota1}\nNota2:  
${nota2}\nNota3: ${nota3}`);  
}  
notas = [3,4,5]  
printNotas(notas)
```

Funções avançadas

- Funções callback
 - Permite-nos executar uma ação após, por exemplo, recebermos os dados necessários;
 - Callback na realidade é uma função passada como argumento para outra função.

```
function myDisplayer(some) {  
    document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2, myCallback) {  
    let sum = num1 + num2;  
    myCallback(sum);  
}  
  
myCalculator(5, 5, myDisplayer);
```

Este conceito é geralmente utilizado quando usamos funções assíncronas!

Funções avançadas

- Factory Functions
 - É uma função que retorna um objeto.

```
function addPerson(firstName, lastName) {  
  return {  
    firstName: firstName,  
    lastName: lastName,  
    getFullName() {  
      return `${this.firstName} ${this.lastName}`;  
    },  
    get message() { // simula que message é um atributo  
      return "Hello! Welcome do this course!";  
    }  
  };  
}  
  
const p1 = addPerson('Pedro', 'Mendonça');  
const p2 = addPerson('Carla', 'Alves');  
console.log(p2.firstName);  
console.log(p1.getFullName());  
console.log(p2.getFullName());  
console.log(p2.message);
```

Funções avançadas

- Factory Functions
 - Podemos definir os sets e gets, por forma a proteger os nossos atributos.

```
function addPerson(firstName, lastName) {  
  return {  
    firstName: firstName,  
    lastName: lastName,  
    get fullName() {  
      return `${this.firstName} ${this.lastName}`;  
    },  
    set fullName(fullName) {  
      fullName = fullName.split(" ");  
      this.firstName = fullName[0];  
      this.lastName = fullName[fullName.length-1];  
    }  
  };  
}  
  
const p1 = addPerson('Pedro', 'Mendonça');  
const p2 = addPerson('Carla', 'Alves');  
console.log(p2.fullName);  
p2.fullName = "Maria Joana Carvalho Freitas";  
console.log(p2.fullName);
```

Funções avançadas

- Função construtora
 - Utilizamos o operador new;
 - Não retorna um objeto;
 - Temos algumas diferenças de sintaxe relativamente às factory functions;
 - Existem conceitos importantes utilizando este tipo de conceito como herança, entre outros.

```
function Person(name, lastName) {  
    this.name = name;  
    this.lastName = lastName;  
  
    this.fullName = function() {  
        return `${this.name} ${this.lastName}`;  
    }  
}  
  
p1 = new Person("Pedro", "Mendonça");  
p1 = new Person("Carla", "Alves");  
console.log(p1.fullName());
```


Prototype - Herança

```
function Product(name, price) {  
  // função construtora  
  this.name = name;  
  this.price = price;  
}  
function Eletronic(name, price, stock) {  
  Product.call(this, name, price); // herda construtor do pai  
  this.stock = stock;  
}  
// define prototype - métodos que serão partilhados pelo objeto pai  
Product.prototype.toString = function() {  
  console.log(`${this.name} PVP: ${this.price}€`);  
}  
Product.prototype.discount = function(val) {  
  if(val < this.price) this.price -= val;  
}  
Eletronic.prototype = Object.create(Product.prototype);  
  
p1 = new Product("some", 123);  
phone = new Eletronic("S24", 1450, 1000);  
phone.discount(155);  
p1.toString()  
phone.toString()
```

- Protótipos são o mecanismo pelo qual os objetos JavaScript herdam recursos uns dos outros.

CONTEÚDOS

- Introdução ao Javascript
- Princípios básicos de JS
- Funções Avançadas
- **Javascript assíncrono**
- Fetch API

JS - Assíncrono

- Promises
 - Promises são um padrão de desenvolvimento que visa representar a conclusão de operações assíncronas;
 - O JavaScript por si só é tido como uma linguagem que tem que lidar com várias chamadas e execuções que não acontecem no momento que o programador executou o código.

```
function sendMessage(msg, time) {  
    setTimeout(() => {  
        console.log(msg)  
    }, time);  
}  
  
sendMessage("Hello", 2000)  
sendMessage("You feeling ok?", 1500)  
sendMessage("Bye", 1000)
```

JS - Assíncrono

- Promises
 - A vantagem do uso de assincronismo é óbvia: ao executar tarefas em paralelo vamos acelerar o nosso código;
 - Temos é que ter o cuidado de “esperar” pelos dados para executar uma determinada ação;
 - Exemplo: Para mostrar os dados de uma query na BD tenho que aguardar que estes sejam entregues.

JS - Assíncrono

- Promises
 - exemplo

```
function sendMessage(msg, time) {  
    // vamos fazer o wrapper com um Promise  
    return new Promise((resolve, reject) => {  
        if(typeof msg !== 'string') reject(new  
Error("Dados inválidos!"))  
        setTimeout(() => {  
            resolve(msg) //resolver a promise:  
sucesso  
        }, time);  
    });  
}
```

```
//then refere-se ao resolve e catch ao reject  
sendMessage("Hello", 1000)  
    .then(response => {  
        console.log(response)  
        return sendMessage("The end", 500)  
    })  
    .then(response => {  
        console.log(response)  
    })  
    .catch(error => {  
        console.log(error)  
    })
```

JS - Assíncrono

- Async await
 - Podemos substituir o uso de then por async await

```
function sendMessage(msg, time) {  
    // vamos fazer o wrapper com um Promise  
    return new Promise((resolve, reject) => {  
        if(typeof msg !== 'string')  
            reject(new Error("Dados inválidos!"))  
        setTimeout(() => {  
            resolve(msg) //resolver a  
promise: sucesso  
        }, time);  
    });  
}
```

```
async function execute() {  
    try {  
        let response = await sendMessage("Hello", 1000)  
        console.log(response)  
        response = await sendMessage("123", 500)  
        console.log(response)  
    } catch (error) {  
        console.log(error)  
    }  
}  
  
execute()
```

CONTEÚDOS

- Introdução ao Javascript
- Princípios básicos de JS
- Funções Avançadas
- Javascript assíncrono
- **Fetch API**

FETCH API

- HTTP requests (FETCH API)
 - Requisições a APIs;
 - Podemos utilizar com Promises ou async/await;
 - Podemos definir o method, body e headers;
 - Muito utilizado em frameworks js: React, Vue e Angular.
- Vamos ver um exemplo utilizando dados do site <https://dummyjson.com/>;
- Selecciona o endpoint <https://dummyjson.com/docs/products> limitado a 10 produtos

FETCH API

- O nosso endpoint: <https://dummyjson.com/products?limit=10>

```
fetch('https://dummyjson.com/products?limit=10')
  .then(response => {
    if(response.ok) {
      return response.json();
    }
  })
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error))
```

```
async function fetchProducts(endpoint) {
  try {
    const response = await fetch(endpoint);
    if(response.ok) {
      const data = await response.json();
      console.log(data);
      return data;
    }
  } catch (error) {
    console.error('Error:', error);
  }
}

fetchProducts('https://dummyjson.com/products?limit=10')
  .then(data => {
    console.log(data);
  });
```

FETCH API

- Desafio 1
 - Pretende-se popular uma página web com uma receita que será obtida através do endpoint: <https://dummyjson.com/recipes/1>
 - A página inicial deverá ter o seguinte aspeto visual:
 - Quando clicamos no botão a receita deverá ser impressa num DIV por baixo do botão. Deverão ser visualizadas as seguintes informações: name, ingredients, instructions, cookTimeMinutes, difficulty, cuisine e image (deverá ser apresentada a imagem)

Receita individual

Carregar receita

FETCH API

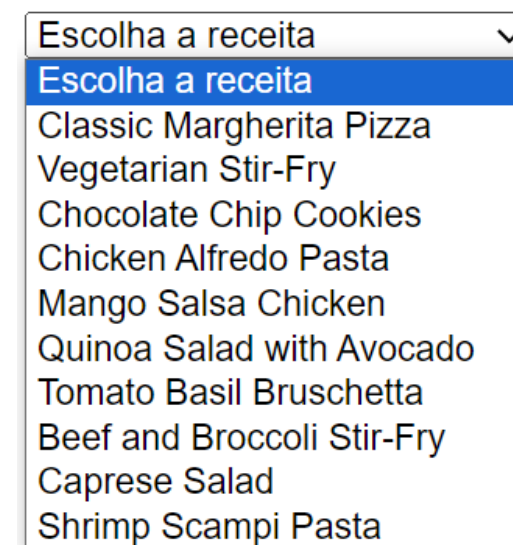
- Desafio 2

- Vamos alterar a nossa página do desafio 1. Em vez de termos um botão vamos adicionar um select que será preenchido com as receitas que vêm do endpoint:

<https://dummyjson.com/recipes?limit=10&select=name>

- Inicialmente a opção selecionada deverá ser “Escolha a receita”;
- Nota: Ao adicionar cada item ao select deve ser mostrado o nome da receita e guardado como value o seu id.

Receita individual



Escolha a receita ▼

Escolha a receita

Classic Margherita Pizza

Vegetarian Stir-Fry

Chocolate Chip Cookies

Chicken Alfredo Pasta

Mango Salsa Chicken

Quinoa Salad with Avocado

Tomato Basil Bruschetta

Beef and Broccoli Stir-Fry

Caprese Salad

Shrimp Scampi Pasta

FETCH API

- Desafio 2
 - Utilizando o id de cada receita no select podemos fazer o pedido à API passando o mesmo como parâmetro como realizado no desafio 1;
 - Adicione o evento “change” quando o value do option é diferente de 0.

Receita individual

Tomato Basil Bruschetta ▾

Tomato Basil Bruschetta

Cuisine: Italian

Difficulty: Easy

Cook Time: 10 minutes

Ingredients:

- Baguette, sliced
- Tomatoes, diced
- Fresh basil, chopped
- Garlic cloves, minced
- Balsamic glaze
- Olive oil
- Salt and pepper to taste

Instructions:

1. Preheat the oven to 375°F (190°C).
2. Place baguette slices on a baking sheet and toast in the oven until golden brown.
3. In a bowl, combine diced tomatoes, chopped fresh basil, minced garlic, and a drizzle of olive oil.
4. Season with salt and pepper to taste.
5. Top each toasted baguette slice with the tomato-basil mixture.
6. Drizzle with balsamic glaze before serving.

AXIOS

- Alternativa ao Fetch API:
npm install axios
- O código é mais simples e “limpo”;
- Automaticamente faz o parse das respostas;

```
const axios = require('axios');

axios.get('https://api.example.com/data')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.error('There was a problem with the axios request:', error);
  });
```

Resumo: requests

- Axios:

```
const axios = require('axios');
const endpoint = "https://dummyjson.com/recipes?limit=10";

// get data from endpoint with axios
axios.get(endpoint)
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.log(error);
  });
```

Resumo: requests

- Fetch (then):

```
const endpoint = "https://dummyjson.com/recipes?limit=10";  
  
// get data with fetch API  
fetch(endpoint)  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.log(error));
```

Resumo: requests

- Fetch (async/await):

```
const endpoint = "https://dummyjson.com/recipes?limit=10";

// get data with async/await
async function getData() {
  try {
    const response = await axios.get(endpoint);
    return response.data;
  } catch (error) {
    console.log(error);
  }
}

getData().then(data => console.log(data));
```


AXIOS API

- Desafio 3
 - Altera a resolução do desafio 2 e utiliza o AXIOS;
 - Adiciona o CDN (após a definição do title no HTML):
`<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>`
 - O resultado da página deverá ser exatamente o mesmo do desafio anterior.



www.cesae.pt

